

ESP32-S3 IoT Platform

Unified Firmware & Hardware Specification

One firmware for three products — bloodBOX · fridgeDataLogger · transformersMonitoring

ETERNITY / ONEOPS Platform · consolidates the MQTT contract, PCB function map and firmware concept

Revised 2026-06-10

Document map. **Part I** — cloud interface & MQTT/EMQX contract (§1–§9). **Part II** — hardware: ESP32-S3 PCB function map, offline timekeeping (§10–§11). **Part III** — firmware concept design: architecture, boot, runtime, link FSM, diagnostics, LoRaWAN integration, EOL manufacturing, partition table, transport hysteresis (§12–§21).

Editable diagram sources live alongside this PDF as draw.io files (`esp32-pcb-functions.drawio`, `esp32-firmware-concept.drawio`); every figure here is redrawn natively so the PDF is self-contained.

Part I Cloud interface & MQTT contract

1 Scope and what was corrected

The specification’s end-to-end sequence (§5.6/§5.7) begins at **publish reading** and treats the broker as a dumb pipe. That hides the parts of the firmware contract that actually decide field reliability: *how the device authenticates, how presence is tracked, how sessions and commands survive a flaky link, and how firmware is delivered safely*. This document redraws the lifecycle of a single production device against **EMQX** broker semantics (MQTT 5.0) and marks every correction.

Area	Original spec	Corrected — EMQX best practice
Transport & auth	shared / implicit; “returns mTLS cert” but no handshake shown	one-time factory <i>bootstrap</i> token over server-auth TLS, immediately rotated to per-device X.509 client certificate ; steady state is mutual TLS 1.3 verified by EMQX.
Client identity	“initial device_id”	stable, unique <code>mqtt_client_id</code> bound to the certificate CN; duplicate-ID takeover rejected.
Authorization	not stated	EMQX ACL : a device may only publish/subscribe under <code>devices/{id}/#</code> and <code>sensors/{sid}/raw</code> ; cross-device spoofing denied.
Session	not stated	MQTT 5.0 <code>clean_start=0</code> + Session-Expiry , so QoS 1 commands queued during a brief drop are re-delivered on reconnect.
Presence (LWT)	status is “LWT topic”, retain unspecified	Will published QoS 1, retain=1 ; matching birth message on (re)connect clears it, so late subscribers always read correct state.
Config delivery	plain publish	<code>devices/{id}/config</code> published retain=1, QoS 1 ⇒ device gets current config the instant it subscribes.
Heartbeat	QoS 0, retain=0 ✓	kept QoS 0; <code>keepalive=60 s</code> , EMQX dead-peer detection at 1.5× <code>keepalive</code> .
Sensor reading	QoS 1, retain=0 ✓	kept QoS 1 (PUBACK); ingest consumes via an EMQX shared subscription (<code>\$share/ingest/...</code>) for horizontal scale + back-pressure.
OTA	“upgrade to 1.4.2 from <code>artefact_uri</code> ”	signed artefact (SHA-256 + secure-boot), A/B partition, out-of-band HTTPS fetch, <code>ota/progress</code> telemetry, automatic rollback , audit row in <code>device_firmware_history</code> .
Abuse control	none	EMQX flapping detection + per-client message/bandwidth rate limits.

Legend. —→ solid = MQTT control packet / synchronous call ---→ dashed = broker delivery / asynchronous event ▷ dotted = return / acknowledgement.

2 Phase A — Zero-touch provisioning & secure session bring-up

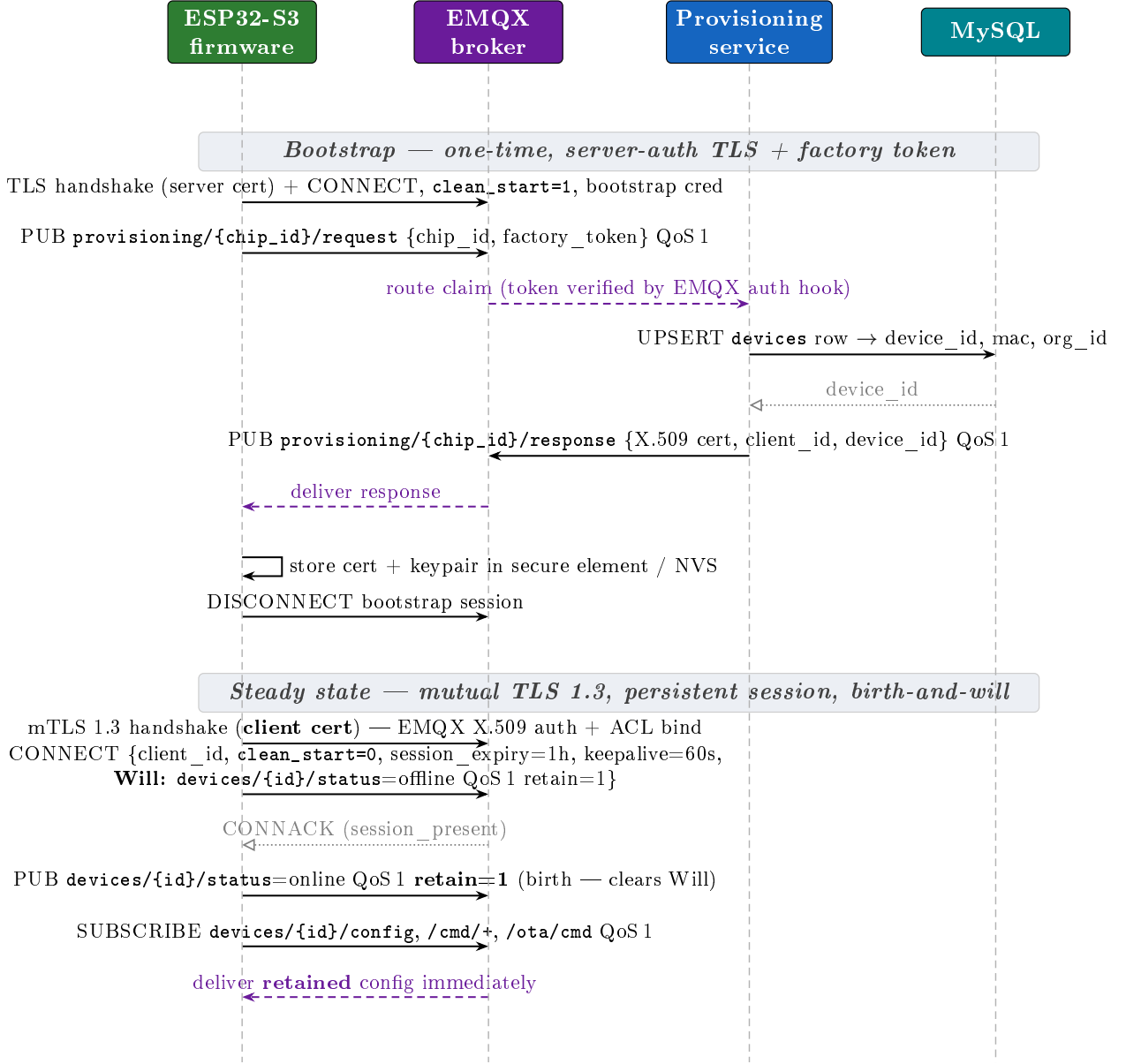


Figure 1: Phase A — provisioning rotates a factory token into a per-device certificate, then the production session is established with mutual TLS, a persistent MQTT 5.0 session, and a retained will/birth pair so presence is never stale.

3 Phase B — Steady-state telemetry (corrected ingestion)

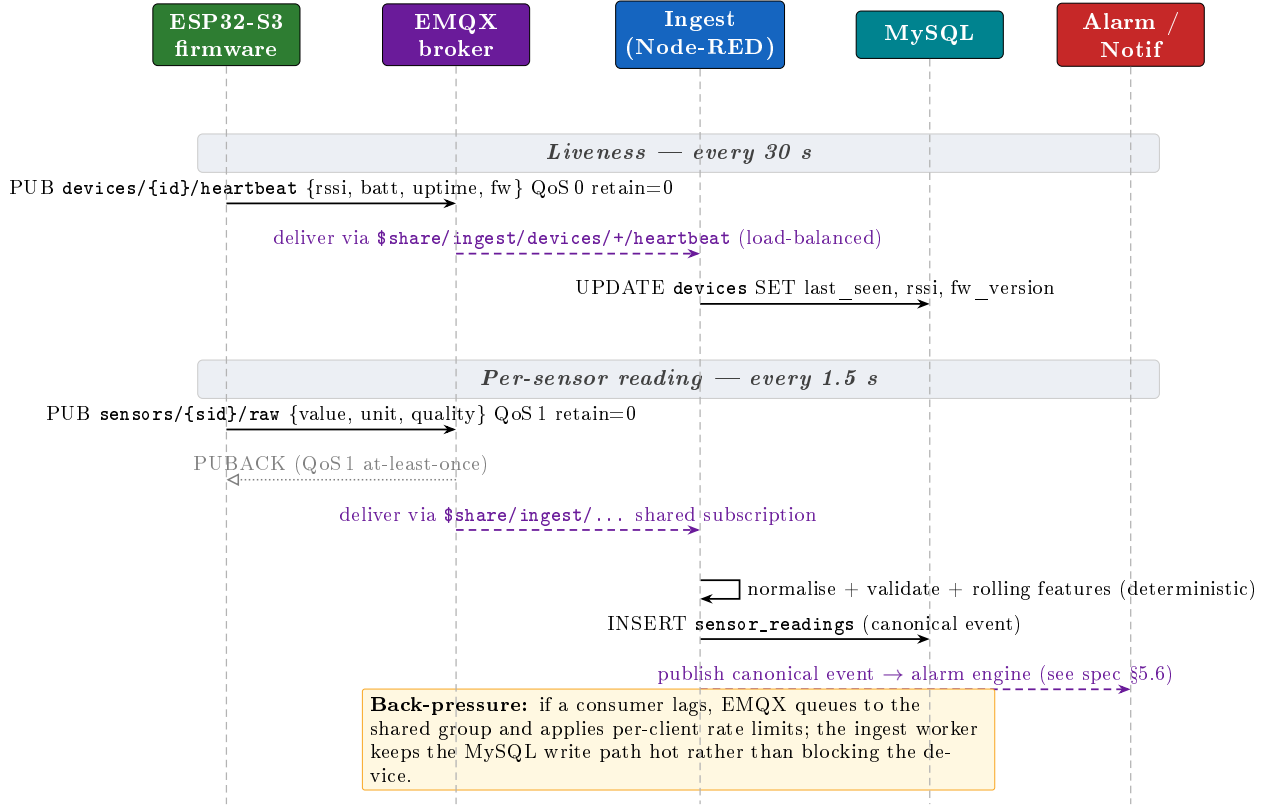


Figure 2: Phase B — heartbeats stay QoS 0; sensor readings stay QoS 1 with PUBACK. The single correction that matters for scale is consuming through an EMQX *shared subscription*, which load-balances across ingest replicas and gives the broker a place to absorb back-pressure.

4 Phase C — Signed OTA update with A/B rollback

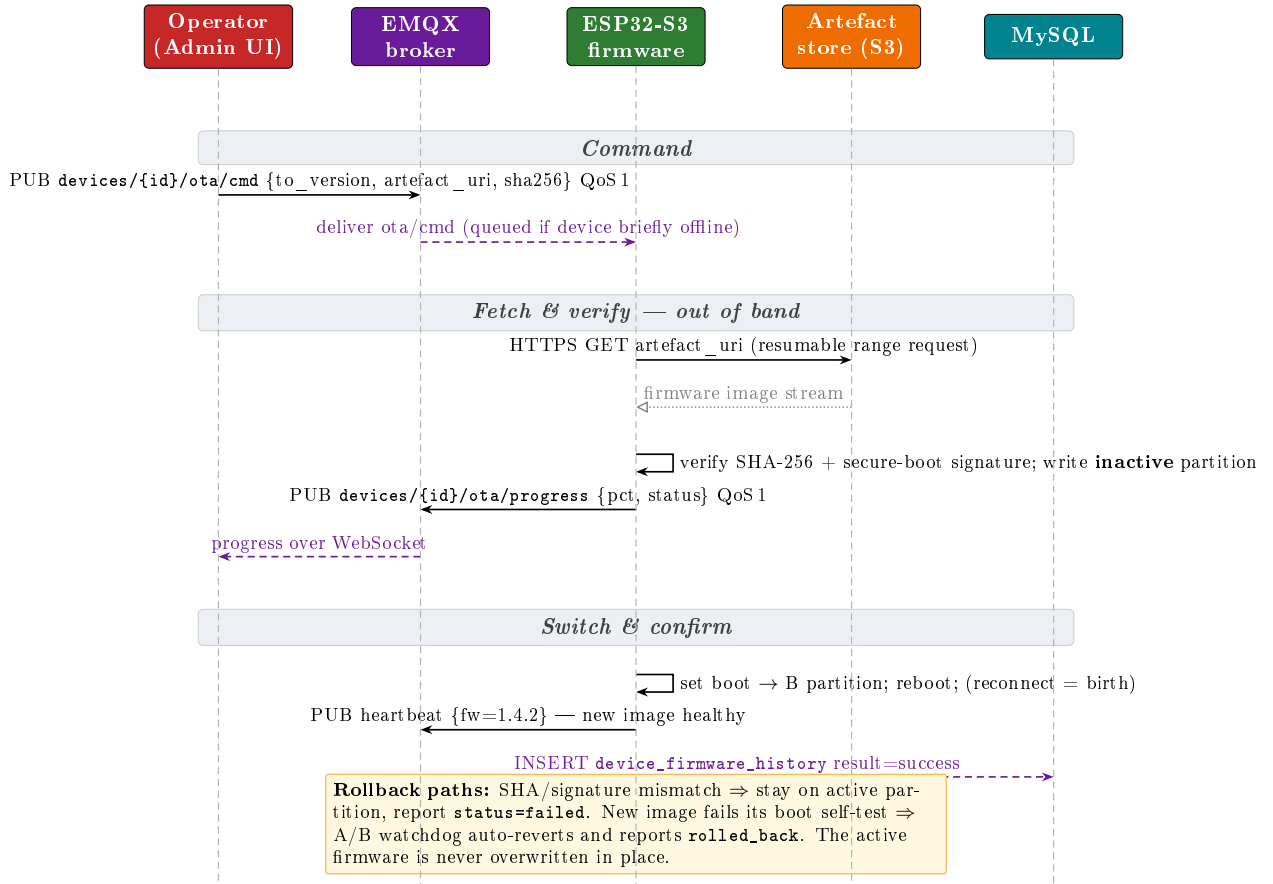


Figure 3: Phase C — OTA is delivered as a signed artefact fetched out of band over HTTPS, flashed to the inactive A/B slot, and only promoted after verification; the firmware version reported in the next heartbeat closes the audit loop.

5 Phase D — Ungraceful disconnect (Last Will & Testament)

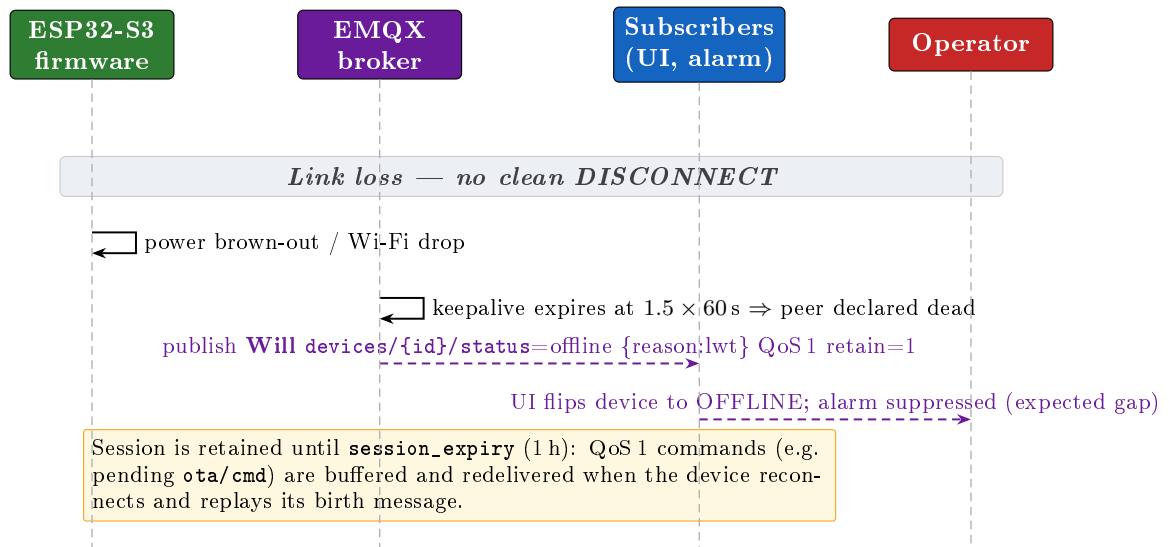


Figure 4: Phase D — because the Will is registered at CONNECT with retain=1, presence stays correct without any device cooperation; the retained session keeps queued commands alive across the outage.

6 Per-product telemetry — one envelope, three payload profiles

The phases above are deliberately product-agnostic because every device speaks the *same* firmware contract. Temperature is transmitted by all three product lines, but it is only *one channel* inside a product-specific set. A device emits **one sensors/{sid}/raw publish per channel per cycle**; the broker and ingest path are identical, and the meaning of each `sensor_id` is resolved by the polymorphic *CTI* model (`sensors.host_fk` $\in$ {t,c,b} $\rightarrow$ exactly one of the three `*_sensor_specs` tables). What actually differs per product is the *channel set*, units, and cadence.

Product line (host)	Cadence		Channels emitted as sensors/{sid}/raw
Refrigeration Data Logger (carbonbox, host=c)	temp 30–60 s;	door on-change	temp_c (°C); door_state (bool 0/1 — <i>event</i> , not a scalar measurement). Cold-chain min/max only.
BloodBOX (host=b)	temp/RH 10–30 s;GPS 1–5 s in transit;impact event-driven		temp_c, rh_pct (humidity), batt_pct, impact_g (shock), gps (lat/lng — <i>compound</i>), baro_alt_m (floor/altitude). Medical transit; richest payload.
Transformer Monitoring (eternity, host=t)	thermal 30–60 s;	DGA minutes–hours	oil_temp_c, ambient_temp_c, dga_h2_ppm (dissolved hydrogen), moisture_pct, oil_level_pct, load_pct. Multi-channel DGA + thermal.

The deltas the generic envelope must absorb: `door_state` and `impact_g` are boolean/event channels mapped as `value` $\in$ {0,1} / `unit=event`; `gps` is a compound channel (sent as two readings `lat`, `lng` or one structured value); cadences span three orders of magnitude (GPS seconds vs. DGA hours). None of this changes the topic or QoS — only the channel set and the spec table the ingest worker joins to.

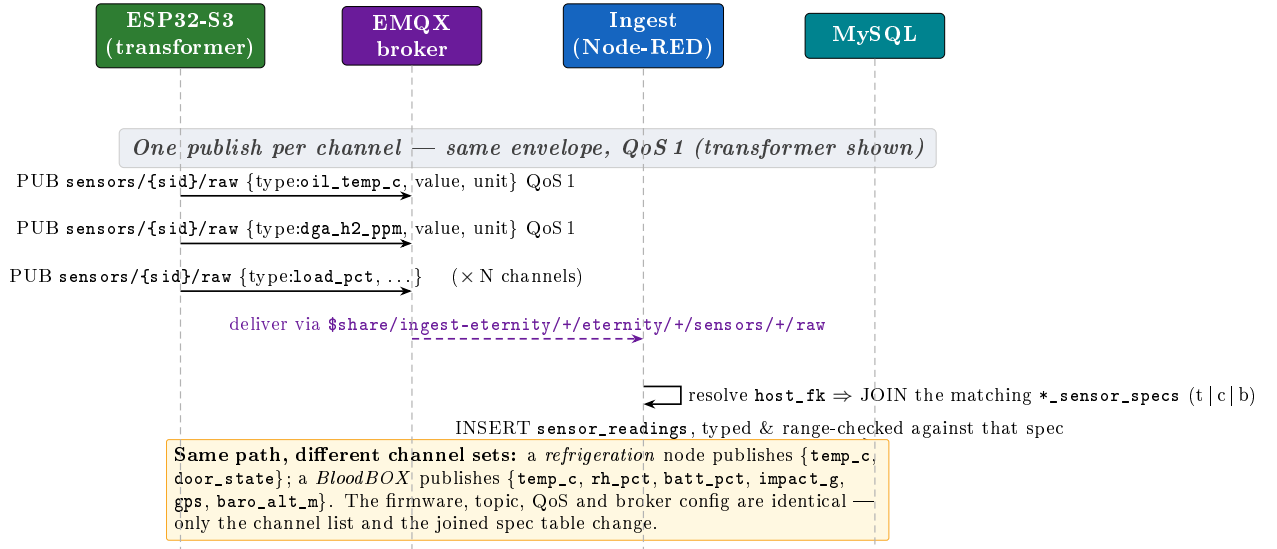


Figure 5: Phase B (per-product) — each device fans out one reading per channel over the single `sensors/{sid}/raw` envelope; the ingest worker resolves the polymorphic host FK and joins the correct per-type spec table, so a 6-channel transformer and a 2-channel fridge share one pipeline.

7 Per-product topic namespace (corrected)

Phases A–D used the device-rooted *tail* of each topic (`devices/{id}/...`, `sensors/{sid}/...`) for legibility. The complete, ACL-enforceable form prefixes **every** production topic with `{tenant}/{product}/{device_id}`, where `product` $\in$ `{eternity, carbonbox, bloodbox}`. The prefix is not cosmetic — it is what lets one broker host three products without cross-talk.

Why product belongs in the topic (EMQX best practice).

- **Isolation & ACL:** the cert CN is `{tenant}.{product}.{device}`; EMQX templates one allow rule `#{cn_tenant}/{cn_product}/{cn_device}/#` and denies everything else. A device physically cannot publish into another product.
- **Per-product ingest pools:** each product runs its own shared-subscription group (`$share/ingest-eternity/...`), because the processing differs — DGA gas-trend models, fridge door-debounce, BloodBOX GPS + excursion timers.
- **Per-product schema, retention, alarm thresholds (§8)** are keyed off the `product` segment with no payload sniffing.
- **Fleet analytics by wildcard:** `+/eternity/#` = every transformer across all tenants (superadmin); `acme/#` = one tenant, all products.

Topic	Dir	QoS	Ret	Notes
<i>Bootstrap listener — pre-identity, flat namespace</i>				
<code>provisioning/{chip_id}/request</code>	D→B	1	0	claim with factory token
<code>provisioning/{chip_id}/response</code>	B→D	1	0	returns X.509 cert + client_id
<i>Production listener — prefix P = {tenant}/{product}/{device_id}</i>				
<code>P/status</code>	D→B	1	1	Will <i>and</i> birth
<code>P/heartbeat</code>	D→B	0	0	30s; liveness & fw_version
<code>P/sensors/{sid}/raw</code>	D→B	1	0	per-channel reading; ingest via \$share
<code>P/alarm/{sid}</code>	B→★	1	1	alarm-engine state, retained (§9)
<code>P/config</code>	B→D	1	1	retained; per-product thresholds (§8)
<code>P/cmd/{op}</code>	B→D	1	0	reboot / calibrate / set-threshold
<code>P/ota/cmd</code>	B→D	1	0	signed artefact descriptor
<code>P/ota/progress</code>	D→B	1	0	percent + status

★ = subscribers: UI (WebSocket bridge), notification service, and optionally the device itself for actuation.

Per-product shared subscriptions & ACL roles.

Role	Authorised topic filter
device (each)	<code>pub+sub {tenant}/{product}/{device}/#</code> only
ingest-eternity	<code>sub \$share/ingest-eternity+/eternity+/sensors+/raw</code>
ingest-carbonbox	<code>sub \$share/ingest-carbonbox+/carbonbox+/sensors+/raw</code>
ingest-bloodbox	<code>sub \$share/ingest-bloodbox+/bloodbox+/sensors+/raw</code>
alarm-engine	<code>pub +/+/+alarm/+;</code> <code>sub +/+/+sensors+/raw</code>
ui / notif	<code>sub {tenant}/#</code> (tenant-scoped, read-only)

8 Per-product alarm thresholds

Thresholds are **not** compiled into firmware. They ship in the retained P/config payload (§7), so the alarm engine and the device's edge fast-path share one source of truth and an operator can retune without an OTA. The values below are taken from the current product logic in `src/lib/`; rows marked (*proposed*) have no hardcoded value yet and are filled with defensible medical/industrial defaults.

Channel	Unit	Warn	Crit	Dir	Guard / debounce
Transformer — eternity (host = t)					
oil_temp_c	°C	80	95	↑	WARN after 2 consecutive readings
ambient_temp_c	°C	40	50	↑	
dga_h2_ppm	ppm	150	300	↑	slow DGA cadence; trend-aware
moisture_ppm	ppm	25	35	↑	
oil_level_pct	%	70	60	↓	inverted (low = bad)
load_pct	%	80	95	↑	
Refrigeration — carbonbox (host = c)					
temp_c	°C	6	8	↑	WARN >6 after 2 readings
door_state	bool	—	open	—	CRITICAL; corrected: debounce >60 s
online	bool	—	offline	—	CRITICAL after 1.5×HB (≈90 s)
BloodBOX — bloodbox (host = b)					
temp_c	°C	6	7	↑	medical: no debounce
excursion	°C	—	> maxTempC (8)	↑	per-transit limit + excursion-time audit
rh_pct	%	80	90	↑	humidity (<i>proposed</i>)
batt_pct	%	40	20	↓	low = bad
impact_g	g	—	3.0	↑	single-sample event ⇒ CRITICAL
lid_state	bool	open	—	—	WARNING on open

Direction: ↑ = high value is bad, ↓ = low value is bad. `door_state/impact_g/lid_state` are event channels (§6).

Retained P/config threshold payloads (one per product).

```
eternity: {"product":"eternity","sample_s":30,"dga_sample_s":3600,
  "thresholds":{
    "oil_temp_c": {"warn":80,"crit":95,"dir":"high","debounce_n":2},
    "ambient_temp_c":{"warn":40,"crit":50,"dir":"high","debounce_n":2},
    "dga_h2_ppm": {"warn":150,"crit":300,"dir":"high"},
    "moisture_ppm": {"warn":25,"crit":35,"dir":"high"},
    "oil_level_pct":{"warn":70,"crit":60,"dir":"low"},
    "load_pct": {"warn":80,"crit":95,"dir":"high"}}}

carbonbox: {"product":"carbonbox","sample_s":30,
  "thresholds":{
    "temp_c": {"warn":6,"crit":8,"dir":"high","debounce_n":2},
    "door_state": {"crit_after_s":60},
    "online": {"crit_after_s":90}}}}

bloodbox: {"product":"bloodbox","sample_s":15,"gps_sample_s":3,
  "thresholds":{
    "temp_c": {"warn":6,"crit":7,"dir":"high"},
    "excursion": {"max_temp_c":8,"audit":true},
    "rh_pct": {"warn":80,"crit":90,"dir":"high"},
    "batt_pct": {"warn":40,"crit":20,"dir":"low"},
    "impact_g": {"crit":3.0,"event":true},
    "lid_state": {"warn_on_open":true}}}}
```

9 Alarm semantics & state machine per product

The alarm engine is the canonical owner of state; the device never decides an alarm row (it may only raise an *edge* hint). State follows the spec §5.5 lifecycle, with these per-product guard differences:

NORMAL → WARNING → CRITICAL → ACKNOWLEDGED → ESCALATED → CLEARED

Product	Guard differences vs. the shared §5.5 lifecycle
eternity	WARNING requires 2 consecutive over-threshold readings; CRITICAL is immediate at $\geq \text{crit. oil_level}$ inverted. One alarm <i>row per channel</i> ; DGA evaluated on a slow cadence with a rising-trend pre-empt.
carbonbox	<code>door_state=open</code> and <code>online=offline</code> bypass the 2-reading debounce $\Rightarrow$ immediate CRITICAL (safety). The corrected design adds a 60s door debounce so routine access does not nuisance-trip.
bloodbox	Temperature is medical $\Rightarrow$ no debounce. The CRITICAL bound is the per-transit <code>maxTempC</code> (mobile, default 8), tracked with an excursion-time audit trail. <code>impact_g</code> is a single-sample event; <code>batt_pct</code> low.

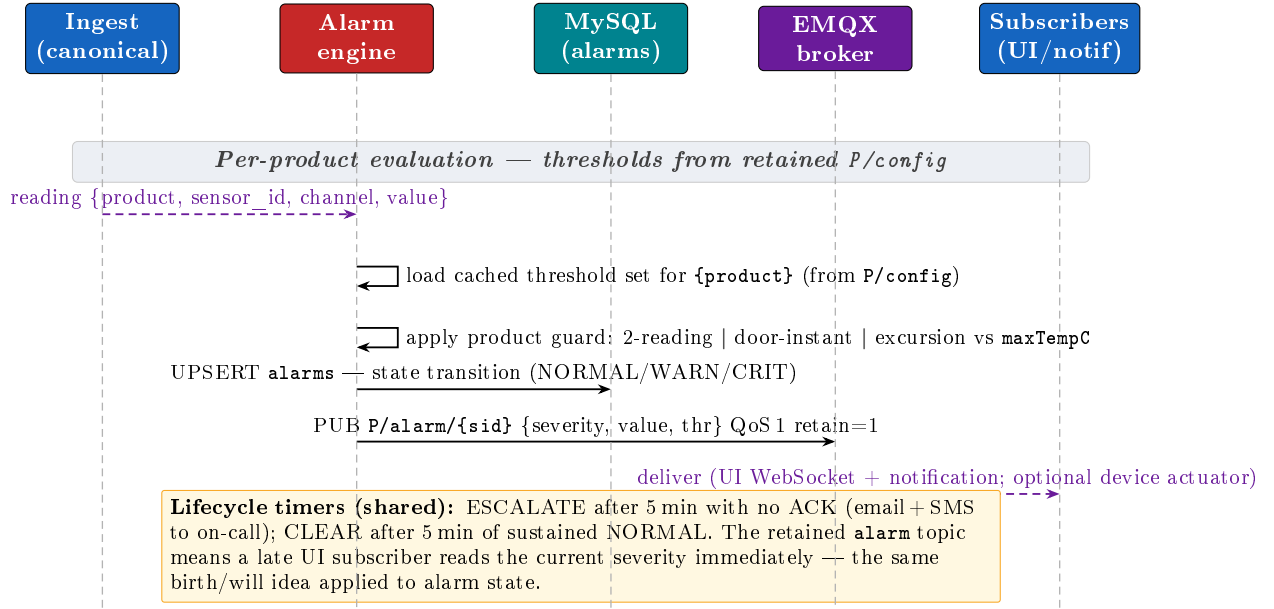


Figure 6: Phase E — one alarm engine, three threshold sets. The reading carries its **product**; the engine loads that product’s thresholds and guard, transitions the **alarms** row per §5.5, and republishes the result on the retained per-device **alarm** topic so presence-of-alarm is never stale.

EMQX configuration checklist.

- Dedicated TLS listeners: a *bootstrap* listener (server-auth, token) and a *production* listener (mutual TLS, `verify=verify_peer, fail_if_no_peer_cert=true`).
- Authentication chain: X.509 CN `{tenant}.{product}.{device}` → `client_id`; reject mismatched or duplicate client IDs.
- Authorization (ACL) templated from the CN so each device is boxed into `{tenant}/{product}/{device}/#` (§7).
- One shared-subscription group *per product* for ingest; alarm engine and UI on their own scoped roles; never a single static subscriber.
- Flapping detection + per-client rate & bandwidth limits enabled.

- Session expiry sized to the longest tolerable command-delivery gap (here 1 h).

Part II Hardware: ESP32-S3 PCB

10 PCB function map

The board is an ESP32-S3-WROOM-1 (N16R8) carrier exposing every field interface the three products need. One firmware drives all of them; which interfaces are *active* is chosen by the product profile (§12).

Functional blocks.

- **MCU core** — ESP32-S3-WROOM-1 (N16R8): 16 MB flash, 8 MB PSRAM, Wi-Fi + BLE; decoupling C2 10 μ F, C10/C4 0.1 μ F.
- **System / boot** — Reset (EN, R13 10 k Ω , RST button), Boot/download (IO0, R20 10 k Ω , PROG button), JTAG (MTDI/MTDO/MTCK).
- **Power** — PWR 12 V in (IN+/IN-), battery B+, rails 12 V $\rightarrow$ 5 V $\rightarrow$ 3.3 V.
- **Communication** — CAN (SN65HVD230, 120 Ω term), RS-485 (MAX3485, 120 Ω term), 4G-UART, LoRa E220-900, USB (native), I²C.
- **Field I/O** — digital inputs (DI), digital outputs (DO1/DO2 @ 12 V), CT (current, ADC), TM1.
- **HMI / storage** — LCD (I²C), SD card, status LEDs (GPIO47 green, GPIO48 red).
- **Timekeeping** — **external RTC (DS3231, TCXO $\pm$ 2 ppm)** on I²C + CR2032 coin cell — holds wall-clock across brown-out and offline windows so every SD-buffered sample and alarm is auditable (§10.1).

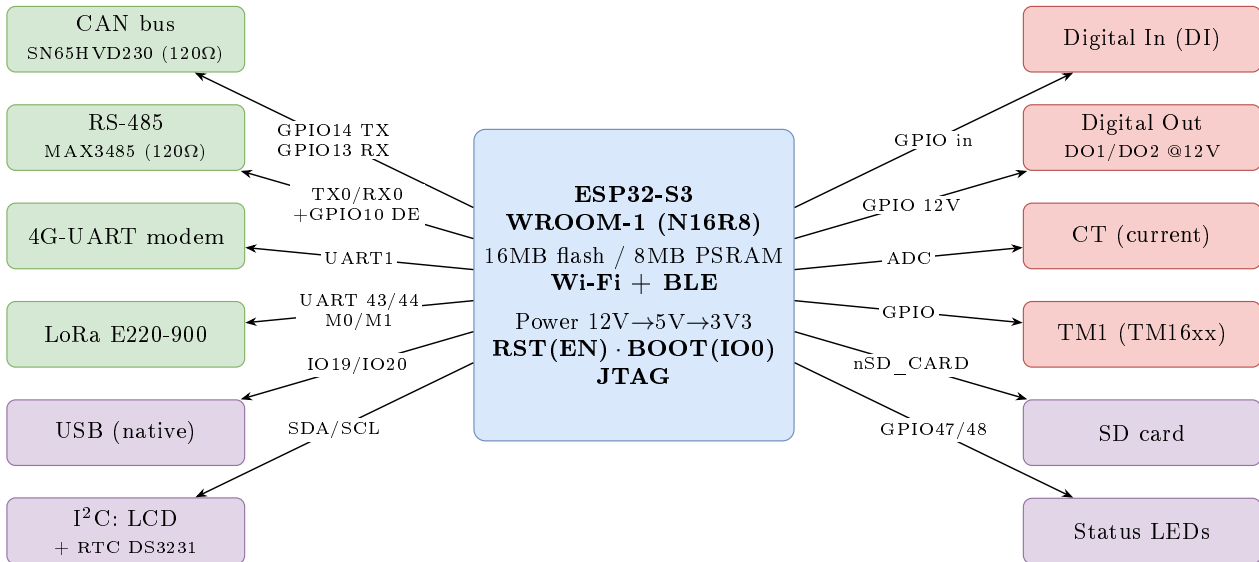


Figure 7: ESP32-S3 PCB — function & signal-flow map (communication left, field-I/O & HMI right; edge labels are the GPIO / signal nets).

Per-product sensor $\rightarrow$ interface.

Product	Channels (PCB interface)
bloodBOX (b)	temp_c(I ² C), rh_pct(I ² C), batt_pct(ADC), impact_g(I ² C accel), gps(UART), baro_alt_m(I ² C)
refrigeDataLogger (c)	temp_c(I ² C/1-wire), door_state(DI, debounced), online(link)
transformersMonitoring (t)	oil_temp_c(RS485/ADC), ambient_temp_c(I ² C), dga_h2_ppm(RS485), moisture_ppm(RS485), oil_level_pct(ADC), load_pct(CT)

Interface assignments are concept-level intent from the PCB map; confirm against the final sensor BOM.

10.1 Offline timekeeping (audit-grade timestamps)

A brown-out reboot with no Internet (4G/Wi-Fi down) would otherwise leave the device with no wall-clock, so SD-buffered rows and the `diag/log` (§16) had to allow `t=null`. The **external RTC** closes that gap: time is seeded from the RTC at boot and *disciplined* whenever a higher-accuracy source appears. Every persisted row carries an absolute timestamp and a `time_src` tag.

Prio	Source	time_src	Accuracy / when
1	NTP over Wi-Fi / 4G	ntp	ms — whenever online
2	LoRaWAN DeviceTimeReq (MAC cmd)	lorawan	< 1 s — LoRa-only coverage (§18)
3	External RTC DS3231 (+ coin cell)	rtc	±2 ppm ($\approx \pm 1$ min/yr) — offline / post brown-out
4	Monotonic uptime only	uptime	relative — pre-sync (<code>t=null</code> , up set)

On every higher-priority acquisition the firmware steps the system clock *and writes it back to the DS3231*, so the RTC stays accurate for the next offline window. This makes `time_src=uptime` a rare boot-race case rather than the normal offline state.

11 GPIO / net map

ESP32-S3 pin	Net	Function
GPIO13	MCU_GPIO_13	CAN RX (SN65 R)
GPIO14	MCU_GPIO_14	CAN TX (SN65 D)
GPIO10	MCU_GPIO_10	RS-485 DE/RE# direction
TXD0 (<i>GPIO43</i>)	MCU_TX0	RS-485 DI / UART0 TX
RXD0 (<i>GPIO44</i>)	MCU_RX0	RS-485 RO / UART0 RX
TX1 (GPIO17) / RX1	MCU_TX1/RX1	UART1 (4G / LoRa)
SDA / SCL	MCU_SDA/SCL	I ² C (LCD)
GPIO19 / GPIO20	MCU_DN(-)/DP(+)	USB D- / D+
GPIO42/41/40	MCU_MTDI/MTDO/MTCK	JTAG
GPIO0	MCU_PROG	boot / download
EN	—	reset
GPIO47 / GPIO48	MCU_GPIO47/48	green / red LED
GPIO45 (<i>inferred</i>)	MCU_nSD_CARD	SD card
GPIO35–38, 06–12	MCU_GPIO_35..38, 06..12	DI / DO / CT / TM1 (<i>field I/O</i>)

Part III Firmware concept design

12 Layered architecture (one firmware, all products)

Everything above the driver layer is product-agnostic. A runtime **product profile** — `{product_id, channel_set, sample_rates, thresholds, transport}` loaded from the retained `.../config` pay-

load (or NVS on first boot) — selects which drivers/channels are active. Adding a product = a profile + drivers; nothing above changes.

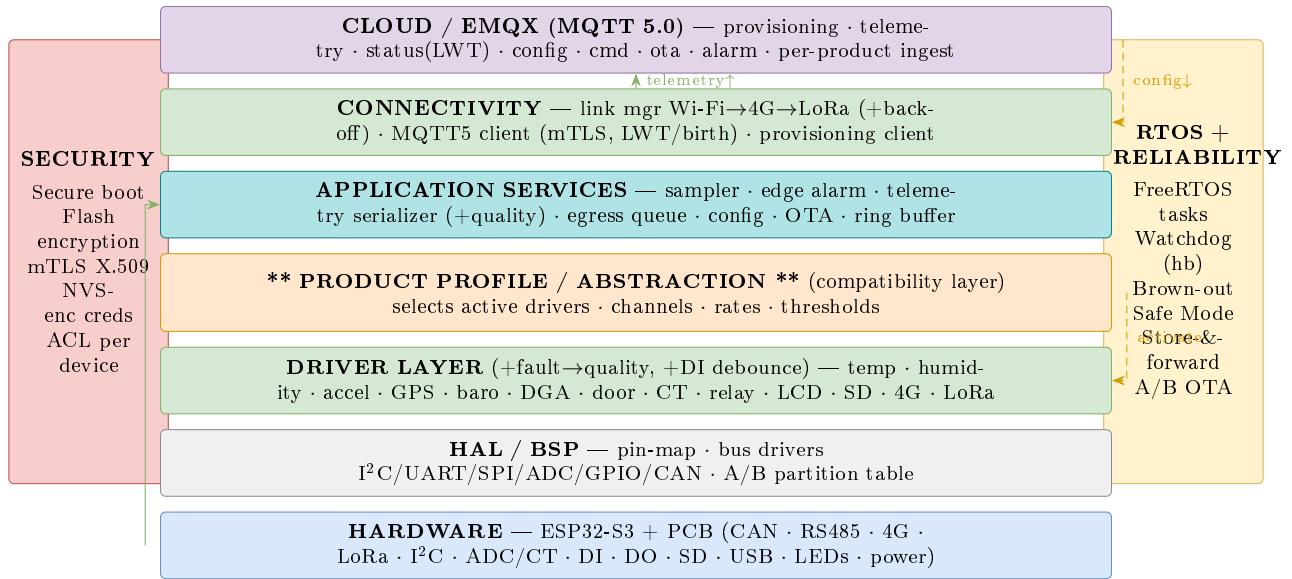


Figure 8: Layered firmware architecture; the highlighted product-profile layer is what makes one image serve three products.

13 Boot & provisioning flow

The happy path is gated by fault checks: a hardware self-test can divert to Safe Mode, and provisioning / link bring-up retry with **exponential backoff (cap + jitter)** so the flow never reaches MQTT connect without a link layer.

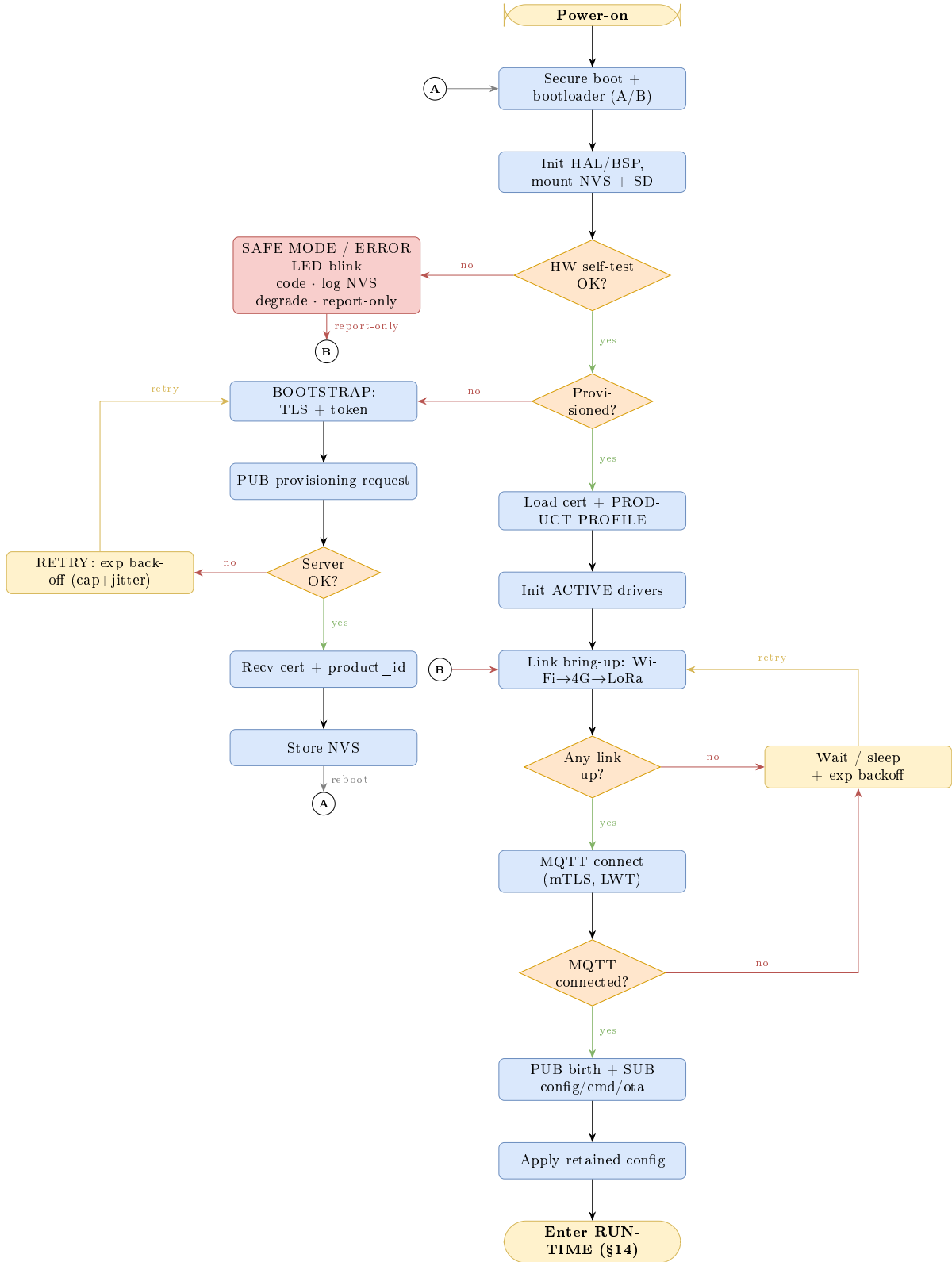


Figure 9: Boot & provisioning with Safe-Mode diversion, bootstrap retry, and link/MQTT backoff loops. Circled **A/B** are on-page connectors (jump to the same-letter target).

14 Runtime tasks & data flow

Sensor and alarm tasks never call publish directly. They enqueue to a single priority **egress queue**; **LinkMgr** is the only publisher (drains one at a time, Alarm QoS 1 before Telemetry), which avoids MQTT-client races and lets the broker absorb back-pressure.

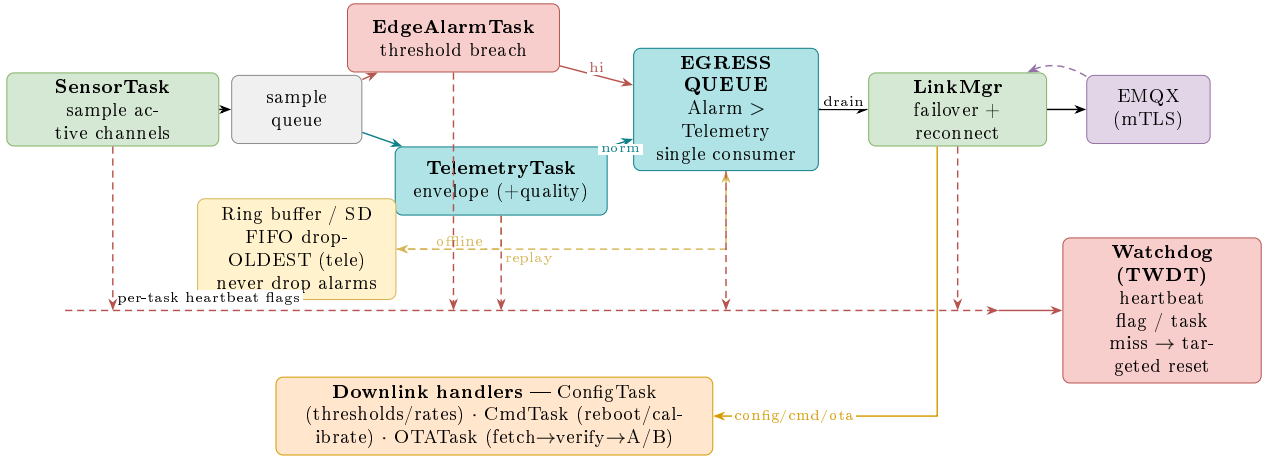


Figure 10: Runtime FreeRTOS tasks, priority egress queue, store-and-forward, downlink handlers and heartbeat watchdog.

15 Link Manager state machine

The FSM below governs the *state* of the active link; *which* transport to select and the anti-flapping rule that prevents needless switching are specified in §21.

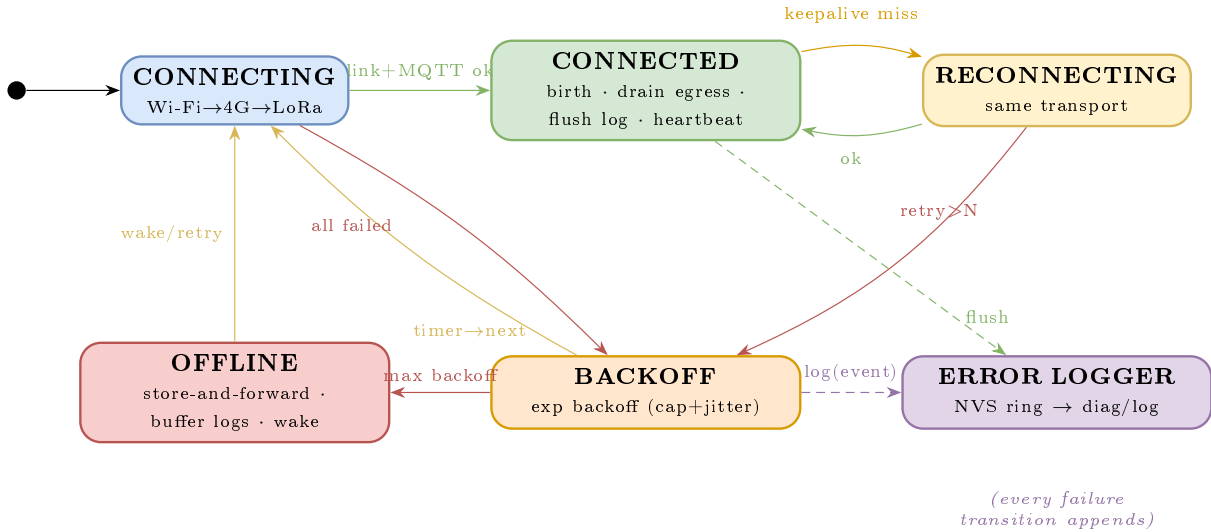


Figure 11: Link Manager FSM: transport failover, backoff, and an error logger that flushes the NVS ring to diag/log when CONNECTED (so offline faults still reach the cloud).

16 Diagnostics: diag/log schema & codes

Topic {tenant}/{product}/{device}/diag/log · QoS 1 · retain=0 · batched flush.

```
{
  "v": 1,                      // schema version
  "device_id": "ac-fridge-0042",
  "tenant": "acme", "product": "carbonbox", // eternity|carbonbox|bloodbox
  "fw": "1.4.2",
  "boot_id": 137,               // ++ each boot (detect resets)
  "sent_at": 1733740800000,     // wall clock at flush (epoch ms)
  "uptime_s": 86230,
  "dropped": 4,                // entries lost to ring overflow
  "events": [
    { "seq": 5012,              // monotonic per device (gap/dedupe)
      "t": 1733740123000,       // epoch ms, or null if pre-time-sync
    }
  ]
}
```

```

    "up": 85553,           // uptime_s when logged (ALWAYS set)
    "time_src": "rtc",     // ntp|lorawan|rtc|uptime (see 10.1)
    "lvl": "warn",        // info|warn|err|fatal
    "code": "LINK_KEEPLIVE_MISS",
    "from": "CONNECTED", "to": "RECONNECTING",
    "tr": "wifi", "rssi": -78,
    "msg": "3 keepalives missed",
    "ctx": { "heap": 142000, "retries": 0 } }
  ]
}

```

Notes. Batched on entering CONNECTED; `t` now comes from the RTC even offline (§10.1) — `time_src` records which clock stamped it; `t=null` only on the rare pre-RTC boot race, where `up` still lets ingest reconstruct wall time as `sent_at - (uptime_s - up)`; `seq` gives gap detection; alarms are *not* sent here (they use `.../alarm/{sid}`).

code	lvl	LED blink	meaning
HW_SD_FAIL	err	2× long	SD mount/format failed
HW_I2C_TIMEOUT	err	3× long	I ² C bus stuck → recover
SENSOR_OPEN_WIRE	warn	2× short	open/loose → quality=error
SENSOR_I2C_NACK	warn	2× short	no ACK → quality=error
SENSOR_STALE	warn	—	no fresh read → quality=stale
PROV_TOKEN_FAIL	err	4× long	factory token rejected
LINK_ALL_FAILED	err	1 Hz	Wi-Fi+4G+LoRa all down
LINK_KEEPLIVE_MISS	warn	—	→ RECONNECTING
LINK_OFFLINE	err	1 Hz	store-and-forward active
LINK_RESTORED	info	—	CONNECTED, flushing log
BUF_OVERFLOW_DROP	warn	—	ring full → drop oldest
OTA_VERIFY_FAIL	err	5× short	SHA / secure-boot mismatch
OTA_ROLLBACK	err	5× short	A/B partition reverted
WDT_RESET	fatal	solid	task hang → targeted reset
BROWNOUT	fatal	solid	power-dip reset

17 Robustness & fault-handling summary

- **Safe Mode** on HW self-test failure (SD/I²C/power): LED error code, NVS fault log, skip the faulty subsystem, attempt a report-only link — never hang.
- **Exponential backoff** (**cap** + **jitter**) on bootstrap, link bring-up and MQTT connect; a link-fallback loop guarantees the flow never enters MQTT `connect` without a link.
- **Priority egress queue** (Alarm QoS 1 > Telemetry) with `LinkMgr` as sole publisher; **ring buffer** policy is FIFO drop-oldest for telemetry, never for alarms.
- **Watchdog** via per-task heartbeat flags → targeted reset of the hung task.
- **Sensor faults** surface as **quality=error:<code>** (never a false 0); **digital inputs** are debounced/rate-limited in the ISR.
- **Diagnostics** (`diag/log`) are flushed when CONNECTED so offline faults still reach the cloud; alarms stay on their own topic.

18 LoRaWAN integration & provisioning

On Wi-Fi / 4G the device speaks MQTT/mTLS to EMQX *directly* (Part I, Phase A). Over LoRaWAN it cannot: the ESP32 transmits compact binary frames to a **LoRa gateway**; a **LoRaWAN Network Server (LNS)** — ChirpStack or The Things Network — terminates the LoRaWAN MAC, runs the OTAA join, de-duplicates and decrypts, then **bridges** decoded uplinks into the *same* EMQX topic namespace. On-air confidentiality is LoRaWAN AES-128 (NwkSKey/AppSKey), not mTLS.

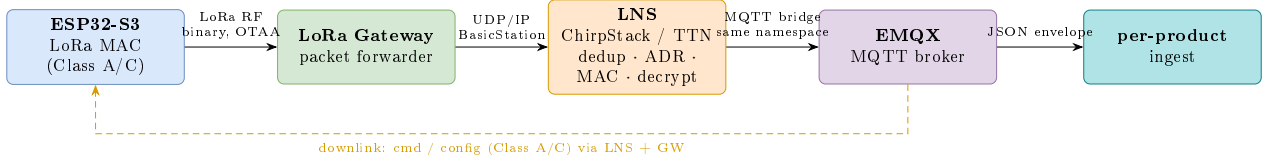


Figure 12: LoRaWAN data path. The gateway + LNS replace the direct MQTT/mTLS hop; the LNS bridge re-publishes decoded frames under `{tenant}/{product}/{device}/...` so the cloud side (ingest, alarm engine) is transport-agnostic.

OTAA credentials (injected at End-of-Line, §19).

Credential	Source (factory)	Stored	Role
DevEUI	chip base-MAC or assigned	eFuse/NVS + label	globally-unique device id on air
JoinEUI (AppEUI)	tenant / application id	encrypted NVS	routes the join to the right join-server
AppKey (root)	per-device, generated at EOL	encrypted NVS (never leaves) + LNS	derives session keys at join

Join (OTAA): `JoinRequest(DevEUI, JoinEUI, DevNonce) → JoinAccept → device derives NwkSKey, AppSKey, DevAddr`. The AppKey and the mTLS certificate are injected *together* at EOL, so one device carries credentials for both transports.

Binary payload (FPort-multiplexed). A LoRaWAN frame is tiny (EU868: ≈ 51 B at DR0 up to 222 B at DR5), so JSON is impossible. Active channels are **batched** into one compact frame; the LNS decoder (or an EMQX rule) re-expands them into the per-channel canonical envelope — this reconciles the “one PUB per channel” of Part I, which applies only on IP transports.

FPort	Message	Dir	Codec	Size	Notes
1	telemetry batch (active channels)	up	CBOR / packed C-struct	8–30 B	per-product channel set
2	heartbeat	up	C-struct	6 B	rss, batt, fw
3	edge alarm	up	C-struct	5 B	high priority, confirmed up
10	diag/log (compact)	up	CBOR	12–40 B	subset of §16
20	cmd (reboot/calibrate/threshold)	down	C-struct	4–12 B	Class A/C
21	config delta (thresholds/rates)	down	CBOR	8–40 B	not full config

Recommendation: CBOR for schema-evolvable messages (telemetry, diag, config) and a **versioned C-struct** (first byte = schema version) for the fixed, size-critical ones (heartbeat, alarm). **Full OTA images are never sent over LoRa** (use Wi-Fi/4G, § Phase C); only small config/threshold pushes go down FPort 21.

19 End-of-Line (EOL) manufacturing & silicon provisioning

eFuse burning is **one-time and irreversible** — a wrong key bricks the chip. The line therefore generates secrets *on-chip*, tests *first*, and burns fuses *last*.

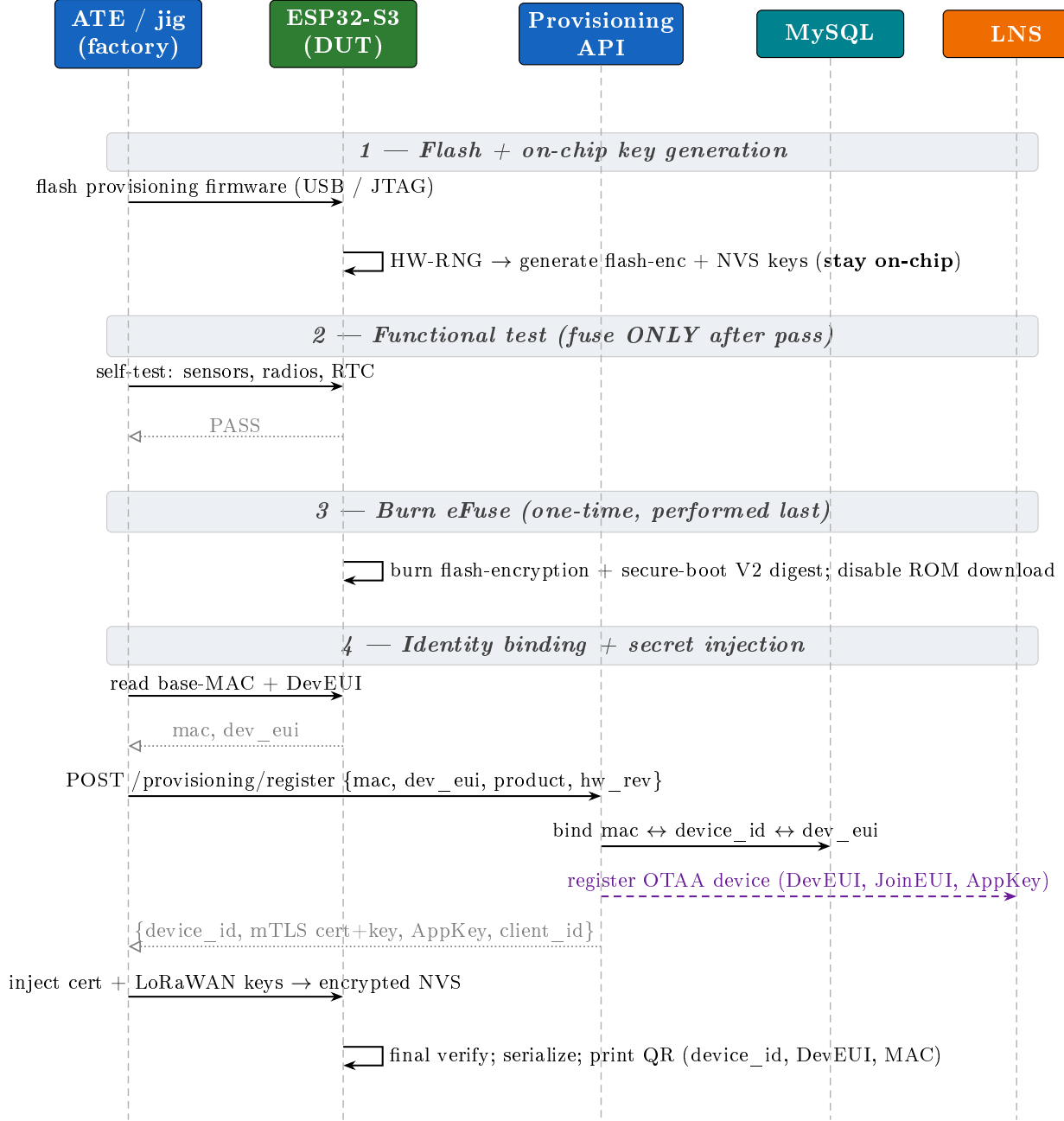


Figure 13: EOL provisioning. Keys are born on the chip and never leave it; the ATE only moves the *public* identity (MAC, DevEUI) to the cloud and injects the per-device cert + LoRaWAN AppKey. Fuses are burned after the functional test to minimise scrap.

MAC → device_id binding API. `POST /api/v1/provisioning/register` returns `{device_id, tenant, mqtt_client_id, cert, app_key}` and writes the immutable `(mac, dev_eui, device_id, product, hw_rev)` tuple to the `devices` table — the same row later joined by the ingest path. The cert CN is `{tenant}.{product}.{device}` (§7).

eFuse safety. Burning is irreversible: (a) dry-run and verify all key digests before the burn; (b) burn as the *last* step, only after every functional test passes; (c) keep the secure-boot *signing* key

in an HSM — only its public digest is fused, never on the production line.

20 Flash partition table (16 MB, frozen at v1)

The A/B OTA scheme requires `ota_0` and `ota_1` to be *equal-sized*; changing any app slot later breaks the bootloader’s slot arithmetic. This table is therefore part of the spec, not a per-build choice.

Name	Type	SubType	Size	Purpose
<code>nvs_keys</code>	data	<code>nvs_keys</code>	4 KB	flash-encryption key store
<code>otadata</code>	data	<code>ota</code>	8 KB	A/B boot selector
<code>phy_init</code>	data	<code>phy</code>	4 KB	RF calibration
<code>nvs</code>	data	<code>nvs</code>	1 MB	mTLS cert, LoRaWAN keys, config
<code>factory</code>	app	<code>factory</code>	3 MB	immutable golden-recovery image
<code>ota_0</code>	app	<code>ota_0</code>	3 MB	OTA slot A
<code>ota_1</code>	app	<code>ota_1</code>	3 MB	OTA slot B
<code>storage</code>	data	<code>littlefs</code>	≈5.9 MB	ring-buffer staging before SD

```
# ESP-IDF partitions-16MB.csv (offsets auto; app partitions 64 KB-aligned)
# Name,      Type, SubType,  Size
nvs_keys,    data, nvs_keys, 0x1000
otadata,     data, ota,     0x2000
phy_init,    data, phy,      0x1000
nvs,         data, nvs,     0x100000
factory,     app,  factory,  0x300000
ota_0,       app,  ota_0,   0x300000
ota_1,       app,  ota_1,   0x300000
storage,     data, littlefs, 0x5F0000
```

If golden-recovery is not required, drop `factory` and grow `storage` by 3 MB — but keep `ota_0 = ota_1`.

21 Transport selection & hysteresis (anti-flapping)

The FSM (§15) handles *losing* a link; this section handles *choosing* one without chattering. Transports are ranked by bandwidth/cost; the device commits to one and resists switching back up until the better link has proven stable — the link-layer analogue of the alarm 2-reading debounce (§8).

Transport	Rank	Switch-up dwell	Switch-down	Notes
Wi-Fi	0 (preferred)	20 s	immediate on loss	cheapest, highest BW; full OTA here
4G / LTE	1	30 s	immediate on loss	metered; full OTA here
LoRaWAN	2 (floor)	—	n/a	always-available fallback; binary only (§18)

- **Switch-down** (to a lower rank) is *immediate* on active-link loss — fast failover, exactly the BACKOFF/OFFLINE paths of §15.
- **Switch-up** (back to a higher rank) only when that link has been *continuously* healthy for its dwell time *and* no publish is in flight. Each link keeps a `stable_since`; the guard is `now - stable_since ≥ dwell`.
- This stops a BloodBOX in transit through patchy 4G from flapping Wi-Fi/4G/LoRa every few seconds and burning power + bandwidth on repeated mTLS handshakes.
- The link being *left* still carries the §15 exponential backoff, so a marginal link is not re-probed faster than its backoff allows.